

Programming Part (200 points)

This assignment will help you gain a practical understanding of *Resampling*, *Quantization* and *Filtering* to understand how these processes affect visual media types like images and video. The motivation here comes from the fact that high valued content is created/saved as a master copy with a high resolution in both samples and color. This master copy may then be appropriately scaled down for distribution depending on the platform. The conversion process needs to ensure that the final product is of the highest quality possible given the constraints. Eg For Digital Cinema, frames are mastered at 4K resolution (4096x2160 pixels) with each pixel at 12 bits per color channel (total 36 bits). For distribution via blue ray/DVD, the image frame may be down sampled to HD format (1920x1080 pixels) with each pixel at 8 bits per channel (total 24 bits). In this assignment we will not use such high resolution but will be starting with moderately high 512x512 images at 24-bit color depth and then further resample/requantize.

First, you need to know how to display images in the RGB format. We have provided sample code to read and display an image in Vscode C++ and java. This source has been provided as a reference for students who may not know how to read and display images. You are free to use this as a start or write your own C, C++, java. *For assignments we do not allow usage of any libraries, nor use scripting environments like python or matlab!*

The input to your program will take four parameters as explained below:

- The first parameter is the name of the image, which will be provided in an 8 bit per channel RGB format (Total 24 bits per pixel). You may assume that all images will be of the same size for this assignment of size 512 x 512. The .rgb file format stores the image in a particular order – first comes the red channel in scan line order, followed by the green and blue channels. Please refer to the reference code if needed. ***Please ensure that in your code you strictly follow the 512x512 resolution, test images will conform to this size.***
- The second parameter will be a **Scale** value. This will control by how much the image must be scaled. It will be a floating-point number greater than 0.0 but less than or equal to 1.0. Effectively, this will result in down sampling your image.
- The third parameter will be a **Quantization** number giving the number of bits used in quantization. The initial rgb file has 24 bits per pixel, which will be quantized down to a lower number. The value of Q will be $1 \leq Q \leq 24$, Q will be a multiple of 3 for the three channels (unless you are attempting the extra credit). Each channel will be quantized to $Q/3$ bits.
- The fourth parameter will suggest a **Mode** for quantization. Its value will be an integer. A $M=-1$ indicates uniform interval quantization, A value $0 < M < 255$ indicates a pivot value for a smarter logarithmic interval quantization. And a $M=256$ indicates a mode for optimal interval quantization. The value of $M=256$ indicates the most optimal interval implementation. All three methods are explained below.
- An optional fifth parameter **E=1** will be used if you indicated in your submission that you have done the extra credit.

How will you be evaluated:

Your submitted code will be compiled, and tests will be run for different scenarios. The evaluation will be comprised of:

1. Resampling tests
2. Quantization tests
3. Analysis Questions
4. Extra Credit (if attempted)

How will you program be invoked:

To invoke your program, we will compile it and run it at the command line as

YourProgram.exe C:/myDir/myImage.rgb S Q M

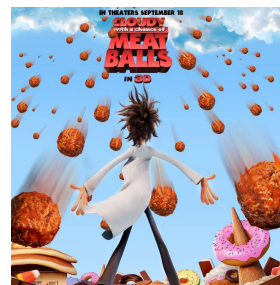
where S Q M are the parameters as described above. Example invocations and their expected outputs are explained below which should give you a fair idea about what your input parameters do and how your program will be tested.

1. *YourProgram.exe myImage.rgb 1.0 24 -1*

Here the image will be scaled by 1.0, with the output quantified to 8 bits per channel and use uniform quantization. With $Q=24$, this indicates that output needs to have 8 bits per channel, which is the same as input channel bits. This effectively means that your output will be the same as your input.

2. *YourProgram.exe myImage.rgb 0.5 24 -1*

Here the image will be scaled by 0.5, at 8 bits per channel. This is the same as the previous case except that the image is scaled down to half its size in each dimension.



3. *YourProgram.exe myImage.rgb 0.8 12 -1*

Here the image will be scaled by 0.8, each channel is being quantized to 4 bits (so a max of 16 values per channel) with uniform quantization. For display reasons you will need to define a mapping for 16 values (for each channel) in the 0-255 range using uniform quantization. For example - 8,24,40,56...etc. are plausible values that correspond to the centers of a uniform range 0-15, 16-31, 32-47, 48-63 ... etc.

4. *YourProgram.exe myImage.rgb 0.75 9 0*

Here the image will be scaled by 0.75, each channel is quantized to 3 bits (so a max of 8 values per channel) with logarithmic quantization. The logarithmic quantization starts at 0. For display reasons you will need to define a mapping for 8 values (for each channel) in the 0-255 range using logarithm quantization. For example, 4, 13, 25, 45, 74...etc. are plausible values that correspond to the centers of a logarithmic range 0-8, 8-17, 17-32, 32-57, 57-92 ... etc.

Details of implementation:

Every pixel has an RGB value and an (x, y) location. When S is not equal to 1.0, you will need to compute the new location of the output pixel (x_{new}, y_{new}) and copy its *filtered* (or *anti-aliased*) RGB value to the new location. The filtered value depends on the filter kernel used and very many kernels are possible. A common practice is to use a uniform 3x3 averaging kernel shown below.

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

Average Filter Kernel

123	133	136
125	135	139
128	138	142

3x3 window at a location

The unfiltered lookup value of the center pixel is 135.

The *average filtered* lookup value of the center pixel is computed using the kernel as $1/9*123 + 1/9*133 + 1/9*136 + 1/9*125 + 1/9*135 + 1/9*139 + 1/9*128 + 1/9*138 + 1/9*142 = 133.222$

You will need to round this value to the closest representation = 133.

Changing the kernel value changes the filter output. When averaging a pixel on the image border or corner, appropriately ignore the undefined pixels in your average computation.

Uniform quantization ($M = -1$)

This mode should be straight forward. Here you take the entire range of 0 – 255 and divide it into uniformly spaced intervals depending on the output number of bits of quantization. For example, if the Q output is 9 (3 bits per channel), you will want to divide each channel's range into 8 intervals. You will then need to assign each range with a representative sample value which is typically the center value of the range. All the values in each range then map to this representative value.

Logarithmic quantization ($M=x$ where x is $0 \leq M \leq 255$)

The motivation to use such non uniform quantization is to appropriately exploit the distribution of your input values. Here we desire to quantize the important pixels with less error and the less important ones with more error. For instance, if your image was